

```
}
common {
    net {
        protocol C;
    }
}
```

The *usage-count yes* option enables collecting real-time statistics of DRBD, and *Protocol C* sets up a *synchronous mode* replication as described earlier in the *Replication Modes* section.

Step 6: Edit */etc/drbd.d/r0.res* on both the nodes:

```
resource r0 {
    device /dev/drbd1;
    disk /dev/sdb1;
    meta-disk /dev/sda2;
    on drbd-server1 {
        address 10.0.6.100:7789;
    }
    on drbd-server2 {
        address 10.0.6.101:7789;
    }
}
```

r0 is our resource name. Here, */dev/drbd1* is the virtual block device to be created by DRBD. The partition */dev/sdb1* holds our replicated data set. The partition */dev/sda2* holds meta information for internal use by DRBD. The port number 7789 is used by DRBD for replication connection, so make sure that it is not in use by any other application running on your system.

Step 7: Clear the meta-disk */dev/sda2* to avoid DRBD configuration issues:

```
sudo dd if=/dev/zero of=/dev/sda2
```

This takes some time, depending on the partition size.

Step 8: Create meta-data for the resource *r0*, as follows:

```
sudo drbdadm create-md r0
```

Step 9: Enable the resource *r0* as follows:

```
sudo drbdadm up r0
```

Step 10: Check the DRBD status in the */proc/drbd* virtual file system, as follows:

```
cat /proc/drbd
```

Check the disk state or *ds* field for 'Inconsistent/Inconsistent', which is normal at this stage since we have not done the initial synchronisation.

Step 11: Start the initial synchronisation from *drbd-server1*, to make it *primary* and other node *secondary*.

```
sudo drbdadm primary --force r0
```

Now, check the status in */proc/drbd* for the ongoing synchronisation. This step takes some time, depending on the disk size. On completion, the disk state *ds* shows 'UpToDate/UpToDate' and roles *ro* shows 'Primary/Secondary'. Here, *primary* indicates that the local node resource is in *primary* mode and the peer is in *secondary* mode. Now, our DRBD set-up is ready for replication.

To test the set-up, let us mount our virtual block device */dev/drbd1* and create a file in it.

Let's assume that the user name and group name of our system are the same, which is *drbd*.

```
mkdir $HOME/safedisk
sudo mount /dev/drbd1 $HOME/safedisk
sudo chgrp drbd $HOME/safedisk
sudo chown drbd $HOME/safedisk
cd $HOME/safedisk
echo "DRBD is up and running!" > myfile
```

Now shut down the *primary* to simulate a failure condition. Go to *secondary* (*drbd-server2*) and run the following command to change it to *primary*:

```
sudo drbdadm primary r0
```

Next, on *drbd-server2*, mount */dev/drbd1* to a local directory as described above and check *myfile* and its content. It should be the same as what we have written from the previous *primary* node.

This is only an introductory article on DRBD. Interested readers are advised to explore further to find out other features, configuration options and use-cases of DRBD. Online training is also available at <http://www.linbit.com/en/p/services/training> by LINBIT. 

References

- [1] <http://drbd.linbit.com/>
- [2] https://en.wikipedia.org/wiki/Distributed_Replicated_Block_Device
- [3] <https://help.ubuntu.com/lts/serverguide/drbd.html>

By: Ramaprasad Chakraborty

The author works at ISRO, Sriharikota, as a scientist engineer SD and has six years of industry experience. He is interested in real-time operating systems, controller redundancy mechanisms, networking, cyber forensics, database management systems and task automation with different levels of scripting and various open source technologies.